SOLID

Object-Oriented Design Principles

S **Single Responsibility Principle**

- A class should have only one reason to change.

O **Open/Closed Principle**

- Open for extension, closed for modification.

L **Liskov Substitution Principle**

- Derived classes should be substitutable for base classes.

I **Interface Segregation Principle**

- No client should be forced to depend on methods it does not use.

D **Dependency Inversion Principle**

- Depend on abstractions, not on concretions.

These aren't random comparisons.

- ✓ Scalability
- ✓ Security
- ✓ Performance
- ✓ Maintainability

Follow @this.tech.girl

& Save and share with dev friends !

@this.tech.girl

S Single Responsibility Principle

What Is It?

- A class should have only one reason to change.

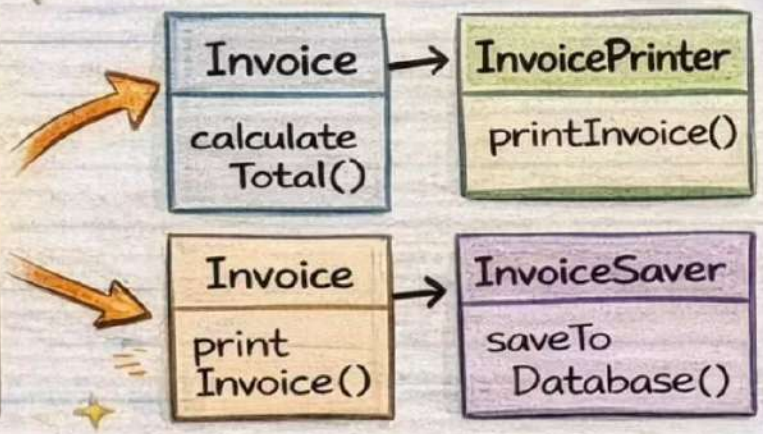
Example:

Before (Violating SRP)

Invoice
(1 class, 3 responsibilities)

- calculateTotal()
- printInvoice()
- saveToDatabase()

Split the Class



Tip: Each class now has one clear responsibility

Interview: SRP helps in maintainability and testability

Follow @this.tech.girl
& Share with dev friends!

@this.tech.girl

Open/Closed Principle

What Is It?

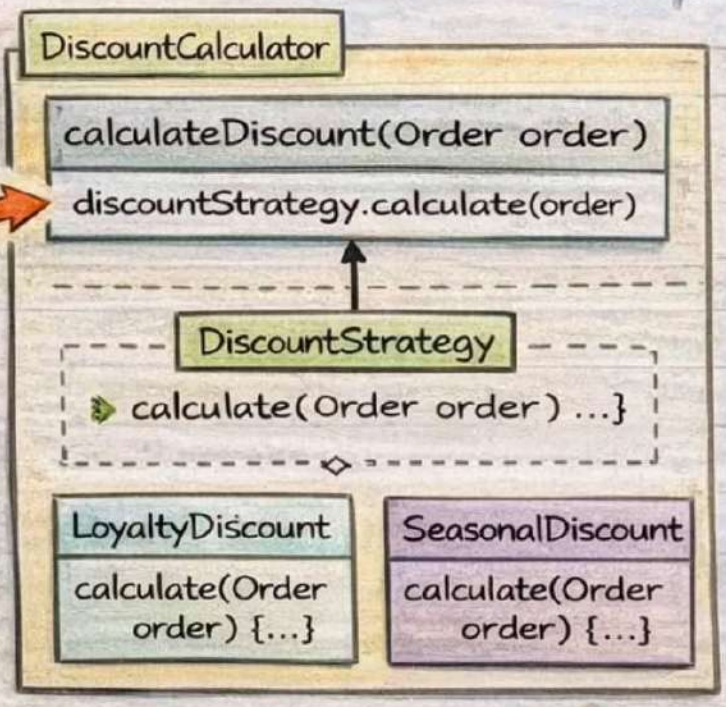
- Software entities should be **open** for extension, but **closed** for modification. ✓ (O) | ✗ |

Example:

Before:

```
DiscountCalculator
calculateDiscount (Order order)
if (order.hasLoyaityPoints() }
   } ... apply loyalty discount ...
}
```

After :



After :

- Discountsrneant options:
  - LoyaltyDiscount
  - SeasonalDiscount
  - Others extend the interface

OCP lets us add new features without breaking existing code.

✓ Interview: Improves maintainability and scalability

✓ Follow @this.tech.girl & Share with dev friends!

Liskov Substitution Principle

What Is It?

- Derived classes should be **substitutable** for their base classes (without breaking functionality).

Example:

Before (Violating LSP)

```
interface: Bird
  fly () // Expected to fly

Penguin implements Bird
  fly ( ) {
  { Can't fly here!
```

After (Following LSP)

```
interface: Bird
calculateDiscount(Order order)
// (prads interfad an malatier'order)

FlyningBird - Flyning - Bird
detinants strategy exendes fly (//)

Penguin:
makeSound( )

Sparrow:
makeSound( )
// Can fly here!
```

Substituing:

```
Bird myBird = new. Penguin()
// Problem!
```

Tip: Penguins can't fly, so separate behaviorsl.

Interview: How does LSP promote polymorphism?

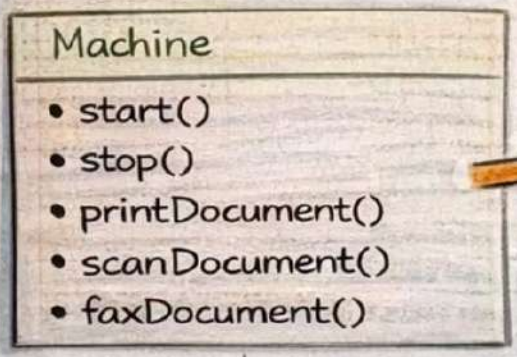
Interface Segregation Principle

What Is It?

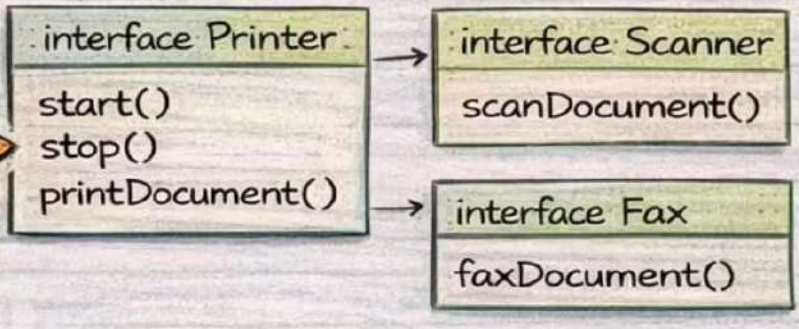
- No client should be forced to depend on methods it does not use

Example:

Before:

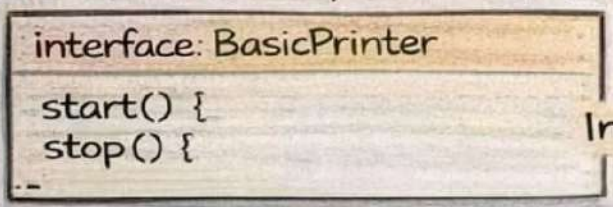


After (Following ISP)



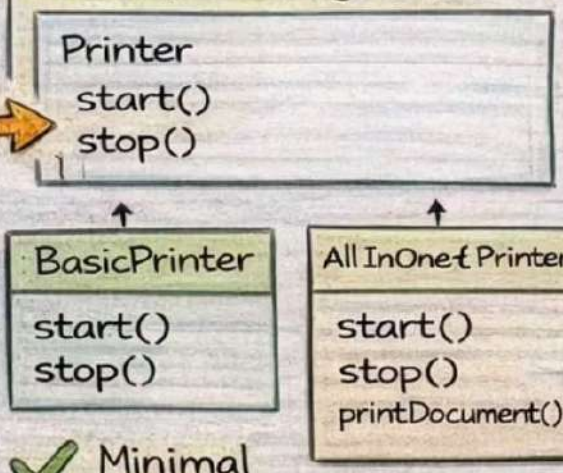
- = Bloated interface ------> Split Interfaces

BasicPrinter implements Machine



Split Interfaces

After (Following ISP)



✓ Minimal Interface

✓ Multi-function

Tip: Small, specific interfaces are easier to understand and mock

☑ Interview: ISP leads to better modularity and flexibility

D Dependency Inversion Principle

What Is It?

- High-level modules should not depend on low-level modules. Both should depend on abstractions.
  - Abstractions should not depend on details.
  - Details should depend on abstractions.

Example:

Wrong Way (Without DIP)

```
class Keyboard
  void type() {
      System.out.printn("Typing...");
  }

class Computer:
Keyboard keyboard = new Keyboard();
  // Direct dependency
}
void start() {
    keyboard.type();
}
```

Correct Way (Using DIP)

```
interface InputDevice
void type();

class Keyboard implements InpitDevice
 public void type() {
      System.out.print("Typing....");
 }

class Computer
private InputDevice inputDevice;
This.input Device = = inputDevice.
void start() {
      faxtDevice.type();
}
```

Tip: Small coupling

- Easily swap dependencies (Keyboard, Mouse, etc.);
- Better testability (mocking devices)

Interview: Improves flexibility and maintainability